Multilayer Perceptron

May 11, 2026

Karolina Antonowicz, Pb 117296, grupa A
Multilayer Perceptron

1. Importowanie bibliotek

W tej części wczytywane są narzędzia niezbędne do pracy z głębokim uczeniem. Biblioteki torch oraz torchvision służą do budowy architektury sieci i obsługi zestawów danych obrazowych. Biblioteki matplotlib i seaborn odpowiadają za wizualizację wyników, a sklearn dostarcza funkcje do oceny jakości modelu, takie jak macierz pomyłek.

[1]:
```
import torch
import torch.nn as nn
import torch.optim as optim
import torchvision
import torchvision.transforms as transforms
from torch.utils.data import DataLoader
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
import pandas as pd
from sklearn.metrics import (
    confusion_matrix,
    classification_report,
    accuracy_score
)
import ssl
ssl._create_default_https_context = ssl._create_unverified_context
```

2. Wczytanie danych i analiza

Projekt wykorzystuje zbiór danych CIFAR-10, składający się z 60 000 kolorowych obrazów w 10 klasach.

Zbiór treningowy liczy 50 000 obrazów, a testowy 10 000.

Obrazy są konwertowane na tensory, a ich piksele zostają znormalizowane do zakresu [0,1]

[2]:
```
transform = transforms.ToTensor()

train_dataset = torchvision.datasets.CIFAR10(
    root="./data",
```

```
    train=True,
    download=False,
    transform=transform
)

test_dataset = torchvision.datasets.CIFAR10(
    root="./data",
    train=False,
    download=False,
    transform=transform
)
```

Model uczy się rozpoznawać obiekty takie jak: samoloty, samochody, ptaki, koty, jelenie, psy, żaby, konie, statki i ciężarówki.

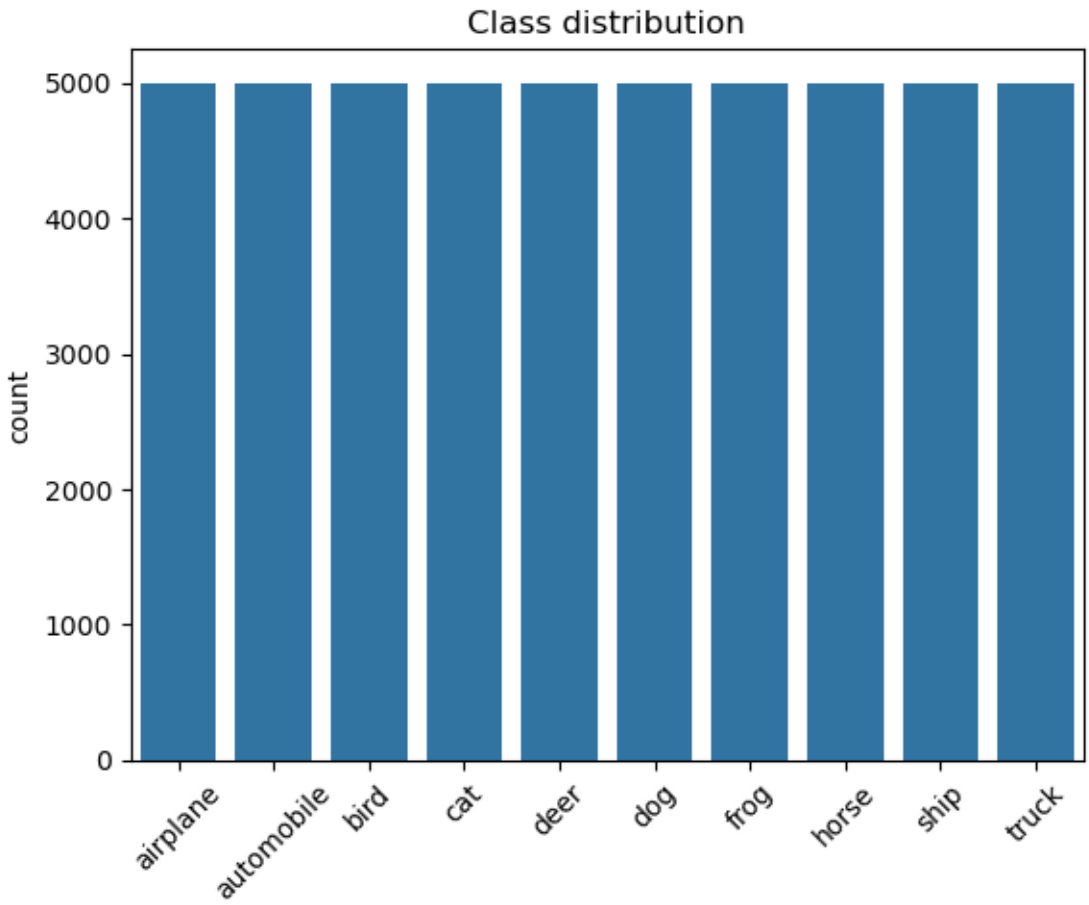
[3]:
```
classes = train_dataset.classes

print("Liczba klas:", len(classes))
print("Klasy:", classes)
print("Rozmiar train:", len(train_dataset))
print("Rozmiar test:", len(test_dataset))
```

```
Liczba klas: 10
Klasy: ['airplane', 'automobile', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse',
'ship', 'truck']
Rozmiar train: 50000
Rozmiar test: 10000
```

[4]:
```
fig, axes = plt.subplots(2, 5, figsize=(12, 5))

for i, ax in enumerate(axes.flat):
    image, label = train_dataset[i]
    image = image.permute(1, 2, 0)
    ax.imshow(image)
    ax.set_title(classes[label])
    ax.axis("off")

plt.tight_layout()
plt.show()
```

Wizualizacja za pomocą countplot potwierdza, że zbiór jest idealnie zbalansowany – każda z 10 klas posiada dokładnie 5 000 przykładów treningowych.

[5]:
```
labels = [label for _, label in train_dataset]

sns.countplot(x=labels)

plt.xticks(range(10), classes, rotation=45)
plt.title("Class distribution")

plt.show()
```



[6]:
```
labels = [label for _, label in train_dataset]
pd.Series(labels).value_counts().sort_index()
```

[6]:
```
0    5000
1    5000
2    5000
3    5000
4    5000
5    5000
6    5000
7    5000
8    5000
9    5000
Name: count, dtype: int64
```

3. Przygotowanie danych

Obliczone statystyki całego zbioru wykazują średnią jasność pikseli na poziomie ok. 0.473 oraz odchylenie standardowe ok. 0.252. Dane są dzielone na paczki (batches) o rozmiarze 64 dla efek-

tywniejszego trenowania.

[7]:
```
all_pixels = torch.cat([
    image.view(-1)
    for image, _ in train_dataset
])

print("Min:", all_pixels.min().item())
print("Max:", all_pixels.max().item())
print("Mean:", all_pixels.mean().item())
print("Std:", all_pixels.std().item())
```

```
Min: 0.0
Max: 1.0
Mean: 0.4733630120754242
Std: 0.2515689432621002
```

[8]:
```
BATCH_SIZE = 64

train_loader = DataLoader(
    train_dataset,
    batch_size=BATCH_SIZE,
    shuffle=True
)

test_loader = DataLoader(
    test_dataset,
    batch_size=BATCH_SIZE,
    shuffle=False
)
```

4. Architektura MLP

Zdefiniowano sieć neuronową o następującej strukturze:
- Warstwa wejściowa: Spłaszczająca obraz o wymiarach 32×32×3 do wektora o długości 3072.
- Warstwy ukryte: Dwie warstwy liniowe (256 i 128 neuronów) z funkcją aktywacji ReLU. - Regularyzacja: Zastosowano warstwy Dropout (0.3), aby zapobiec przeuczeniu modelu.
- Warstwa wyjściowa: 10 neuronów odpowiadających liczbie klas.

[9]:
```
class MLP(nn.Module):

    def __init__(
        self,
        hidden1=256,
        hidden2=128,
        dropout=0.3
    ):

        super().__init__()
```

```
        self.flatten = nn.Flatten()
        self.network = nn.Sequential(
            nn.Linear(32 * 32 * 3, hidden1),
            nn.ReLU(),
            nn.Dropout(dropout),
            nn.Linear(hidden1, hidden2),
            nn.ReLU(),
            nn.Dropout(dropout),
            nn.Linear(hidden2, 10)
        )

    def forward(self, x):
        x = self.flatten(x)
        return self.network(x)
```

[10]:
```
def evaluate_accuracy(model, loader):
    device = next(model.parameters()).device
    model.eval()
    correct = 0
    total = 0

    with torch.no_grad():
        for images, labels in loader:
            images = images.to(device)
            labels = labels.to(device)
            outputs = model(images)
            predictions = outputs.argmax(dim=1)
            correct += (predictions == labels).sum().item()
            total += labels.size(0)
    return correct / total
```

5. Trenowanie i krzywe uczenia

Model trenowano przez 20 epok przy użyciu optymalizatora Adam i funkcji straty CrossEntropyLoss. - Podczas procesu uczenia obserwowany jest systematyczny spadek straty (Loss) i wzrost dokładności (Accuracy). - Wynik końcowy: Model osiągnął dokładność na poziomie ok. 45% w ostatniej epoce.

[11]:
```
def train_model(
    model,
    train_loader,
    test_loader,
    epochs=20,
    lr=0.001
):

    device = torch.device(
        "cuda" if torch.cuda.is_available() else "cpu")
```

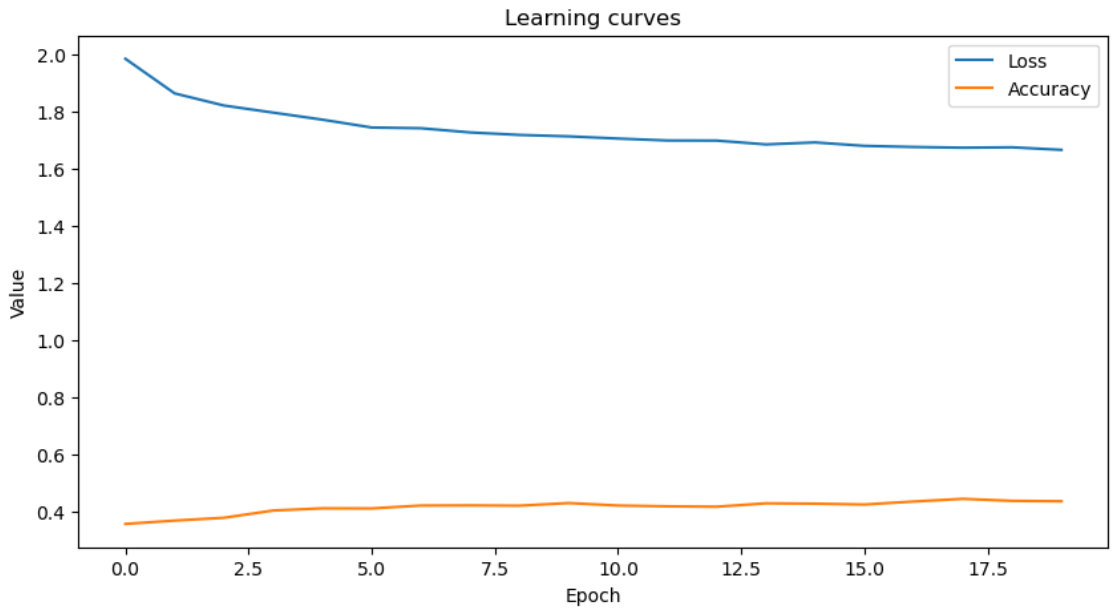
```
    model.to(device)
    criterion = nn.CrossEntropyLoss()
    optimizer = optim.Adam(
        model.parameters(),
        lr=lr
    )

    train_losses = []
    test_accuracies = []

    for epoch in range(epochs):
        model.train()
        running_loss = 0.0
        for images, labels in train_loader:
            images = images.to(device)
            labels = labels.to(device)
            optimizer.zero_grad()
            outputs = model(images)
            loss = criterion(outputs, labels)
            loss.backward()
            optimizer.step()
            running_loss += loss.item()
        avg_loss = running_loss / len(train_loader)

        accuracy = evaluate_accuracy(
            model,
            test_loader
        )

        train_losses.append(avg_loss)
        test_accuracies.append(accuracy)

        print(
            f"Epoch {epoch+1}: "
            f"Loss={avg_loss:.4f}, "
            f"Accuracy={accuracy:.4f}"
        )

    return train_losses, test_accuracies
```

[12]:
```
model = MLP(
    hidden1=256,
    hidden2=128,
    dropout=0.3
)

train_losses, test_accuracies = train_model(
```

```
    model,
    train_loader,
    test_loader,
    epochs=20,
    lr=0.001
)
```

```
Epoch 1: Loss=1.9842, Accuracy=0.3586
Epoch 2: Loss=1.8632, Accuracy=0.3703
Epoch 3: Loss=1.8208, Accuracy=0.3801
Epoch 4: Loss=1.7961, Accuracy=0.4055
Epoch 5: Loss=1.7714, Accuracy=0.4131
Epoch 6: Loss=1.7439, Accuracy=0.4128
Epoch 7: Loss=1.7414, Accuracy=0.4229
Epoch 8: Loss=1.7267, Accuracy=0.4234
Epoch 9: Loss=1.7180, Accuracy=0.4225
Epoch 10: Loss=1.7130, Accuracy=0.4315
Epoch 11: Loss=1.7053, Accuracy=0.4230
Epoch 12: Loss=1.6986, Accuracy=0.4201
Epoch 13: Loss=1.6983, Accuracy=0.4188
Epoch 14: Loss=1.6849, Accuracy=0.4306
Epoch 15: Loss=1.6918, Accuracy=0.4294
Epoch 16: Loss=1.6798, Accuracy=0.4265
Epoch 17: Loss=1.6759, Accuracy=0.4369
Epoch 18: Loss=1.6734, Accuracy=0.4463
Epoch 19: Loss=1.6747, Accuracy=0.4392
Epoch 20: Loss=1.6659, Accuracy=0.4379
```

[13]:
```
plt.figure(figsize=(10, 5))

plt.plot(train_losses, label="Loss")
plt.plot(test_accuracies, label="Accuracy")

plt.xlabel("Epoch")
plt.ylabel("Value")

plt.title("Learning curves")
plt.legend()
plt.show()
```



[14]:
```
all_preds = []
all_labels = []

device = next(model.parameters()).device

model.eval()

with torch.no_grad():

    for images, labels in test_loader:
        images = images.to(device)
        outputs = model(images)
        preds = outputs.argmax(dim=1).cpu()

        all_preds.extend(preds.numpy())
        all_labels.extend(labels.numpy())
```

6. Ocena jakości modelu

- Macierz pomyłek (Confusion Matrix): Pozwala zidentyfikować, które klasy są najczęściej mylone. Model najlepiej radzi sobie z klasami takimi jak “ship” (666 poprawnych) i “frog” (660 poprawnych), a najgorzej z klasą “bird” (tylko 186 poprawnych).
- Raport klasyfikacji: Pokazuje precyzję i czułość dla każdej klasy. Najwyższą precyzję osiągnięto dla samochodów (0.60), a najniższą dla kotów (0.30).
- Analiza błędów: Wyświetlenie błędnie sklasyfikowanych obrazów pokazuje, że sieć często myli wizualnie podobne obiekty, np. psa z żabą lub statki z ciężarówkami

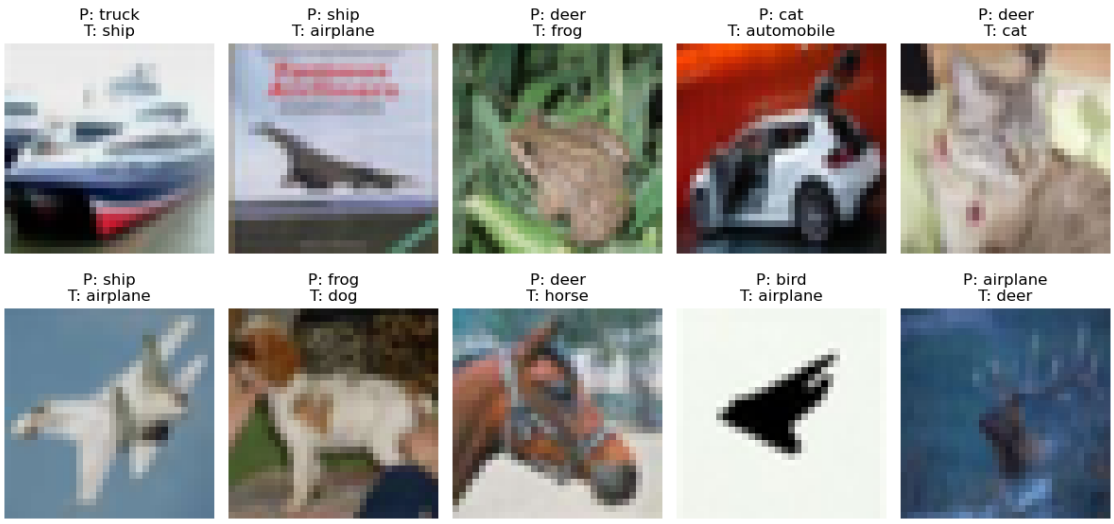
[15]:
```
cm = confusion_matrix(all_labels, all_preds)

plt.figure(figsize=(10, 8))

sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=classes,
    yticklabels=classes
)

plt.xlabel("Predicted")
plt.ylabel("True")
plt.title("Confusion Matrix")
plt.show()
```

[16]:
```
report = classification_report(
    all_labels,
    all_preds,
    target_names=classes
)

print(report)
```

```
              precision    recall  f1-score   support

    airplane       0.64      0.32      0.43      1000
  automobile       0.61      0.53      0.57      1000
        bird       0.37      0.10      0.15      1000
         cat       0.24      0.30      0.27      1000
        deer       0.37      0.44      0.41      1000
         dog       0.39      0.32      0.35      1000
        frog       0.41      0.62      0.50      1000
       horse       0.48      0.52      0.50      1000
        ship       0.55      0.60      0.58      1000
       truck       0.43      0.62      0.51      1000

    accuracy                           0.44     10000
   macro avg       0.45      0.44      0.43     10000
weighted avg       0.45      0.44      0.43     10000
```

[17]:
```
wrong_images = []
wrong_preds = []
wrong_labels = []

model.eval()

with torch.no_grad():

    for images, labels in test_loader:
        images = images.to(device)
        outputs = model(images)
        preds = outputs.argmax(dim=1).cpu()

        for i in range(len(preds)):
            if preds[i] != labels[i]:
                wrong_images.append(images[i].cpu())
                wrong_preds.append(preds[i])
                wrong_labels.append(labels[i])
```

[18]:
```
fig, axes = plt.subplots(2, 5, figsize=(12, 6))

for i, ax in enumerate(axes.flat):
    image = wrong_images[i].permute(1, 2, 0)
    ax.imshow(image)
    ax.set_title(
        f"P: {classes[wrong_preds[i]]}\n"
        f"T: {classes[wrong_labels[i]]}"
    )
    ax.axis("off")

plt.tight_layout()
plt.show()
```



7. Eksperymenty i optymalizacja

- Learning Rate: Najlepsze wyniki uzyskano dla małych wartości (0.0001). Zbyt wysoki parametr (0.1) powoduje brak zdolności modelu do nauki.

[19]:
```
learning_rates = [0.0001, 0.001, 0.01, 0.1]
results = {}
```

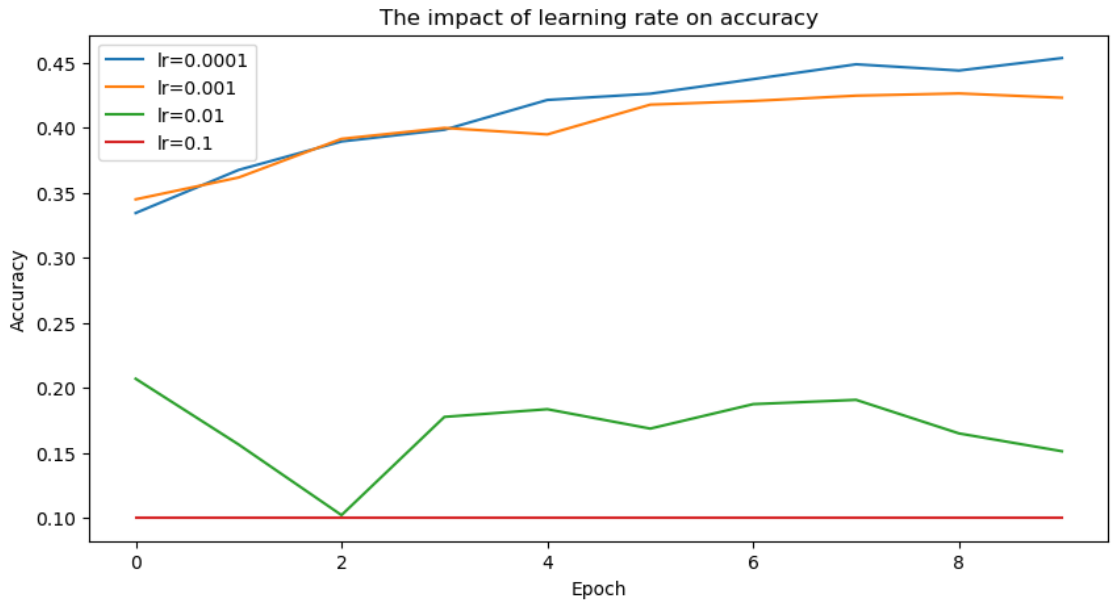
[20]:
```
for lr in learning_rates:
    print(f"\nTraining model with lr={lr}")

    model = MLP(
        hidden1=256,
        hidden2=128,
        dropout=0.3
    )
```

```
    losses, accuracies = train_model(
        model,
        train_loader,
        test_loader,
        epochs=10,
        lr=lr
    )

    results[lr] = {
        "losses": losses,
        "accuracies": accuracies,
        "final_accuracy": accuracies[-1]
    }
```

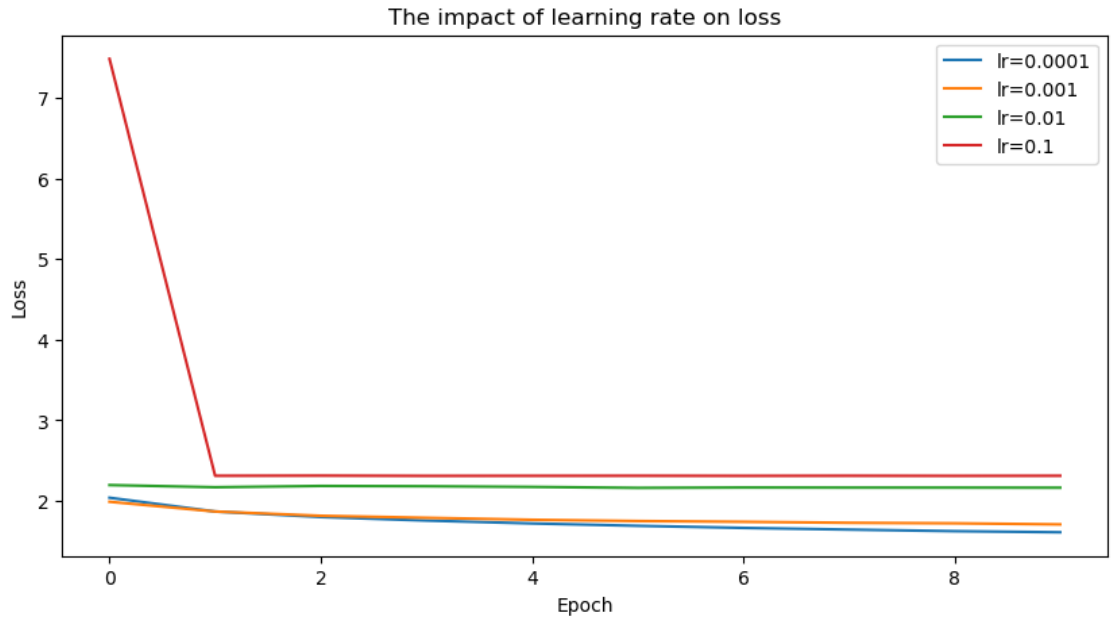
```
Training model with lr=0.0001
Epoch 1: Loss=2.0397, Accuracy=0.3344
Epoch 2: Loss=1.8687, Accuracy=0.3675
Epoch 3: Loss=1.8001, Accuracy=0.3894
Epoch 4: Loss=1.7564, Accuracy=0.3984
Epoch 5: Loss=1.7208, Accuracy=0.4213
Epoch 6: Loss=1.6914, Accuracy=0.4261
Epoch 7: Loss=1.6652, Accuracy=0.4373
Epoch 8: Loss=1.6445, Accuracy=0.4487
Epoch 9: Loss=1.6248, Accuracy=0.4439
Epoch 10: Loss=1.6113, Accuracy=0.4535

Training model with lr=0.001
Epoch 1: Loss=1.9895, Accuracy=0.3449
Epoch 2: Loss=1.8685, Accuracy=0.3617
Epoch 3: Loss=1.8157, Accuracy=0.3914
Epoch 4: Loss=1.7908, Accuracy=0.3998
Epoch 5: Loss=1.7661, Accuracy=0.3948
Epoch 6: Loss=1.7504, Accuracy=0.4177
Epoch 7: Loss=1.7413, Accuracy=0.4205
Epoch 8: Loss=1.7286, Accuracy=0.4246
Epoch 9: Loss=1.7225, Accuracy=0.4263
Epoch 10: Loss=1.7090, Accuracy=0.4230

Training model with lr=0.01
Epoch 1: Loss=2.1965, Accuracy=0.2070
Epoch 2: Loss=2.1697, Accuracy=0.1565
Epoch 3: Loss=2.1854, Accuracy=0.1022
Epoch 4: Loss=2.1820, Accuracy=0.1777
Epoch 5: Loss=2.1737, Accuracy=0.1836
Epoch 6: Loss=2.1617, Accuracy=0.1687
Epoch 7: Loss=2.1661, Accuracy=0.1875
```

```
Epoch 8: Loss=2.1649, Accuracy=0.1908
Epoch 9: Loss=2.1648, Accuracy=0.1650
Epoch 10: Loss=2.1637, Accuracy=0.1513

Training model with lr=0.1
Epoch 1: Loss=7.4803, Accuracy=0.1000
Epoch 2: Loss=2.3118, Accuracy=0.1000
Epoch 3: Loss=2.3134, Accuracy=0.1000
Epoch 4: Loss=2.3106, Accuracy=0.1000
Epoch 5: Loss=2.3113, Accuracy=0.1000
Epoch 6: Loss=2.3120, Accuracy=0.1000
Epoch 7: Loss=2.3109, Accuracy=0.1000
Epoch 8: Loss=2.3119, Accuracy=0.1000
Epoch 9: Loss=2.3107, Accuracy=0.1000
Epoch 10: Loss=2.3118, Accuracy=0.1000
```

[21]:
```python
plt.figure(figsize=(10, 5))

for lr in learning_rates:
    plt.plot(
        results[lr]["accuracies"],
        label=f"lr={lr}"
    )

plt.xlabel("Epoch")
plt.ylabel("Accuracy")

plt.title("The impact of learning rate on accuracy")

plt.legend()

plt.show()
```



[22]:
```
plt.figure(figsize=(10, 5))

for lr in learning_rates:
    plt.plot(
        results[lr]["losses"],
        label=f"lr={lr}"
    )

plt.xlabel("Epoch")
plt.ylabel("Loss")

plt.title("The impact of learning rate on loss")

plt.legend()

plt.show()
```



[23]:
```
summary = []

for lr in learning_rates:
    summary.append({
        "Learning Rate": lr,
        "Final Accuracy": results[lr]["final_accuracy"]
    })

df_results = pd.DataFrame(summary)

df_results
```

[23]:

	Learning Rate	Final Accuracy
0	0.0001	0.4535
1	0.0010	0.4230
2	0.0100	0.1513
3	0.1000	0.1000

- Batch Size: Większe paczki danych (128) zapewniły nieco wyższą stabilność i wynik końcowy w porównaniu do bardzo małych paczek (16).

[24]:
```
batch_sizes = [16, 32, 64, 128]
batch_results = {}
```

[25]:
```
for batch_size in batch_sizes:

    print(f"\nTraining model with batch size = {batch_size}")
```

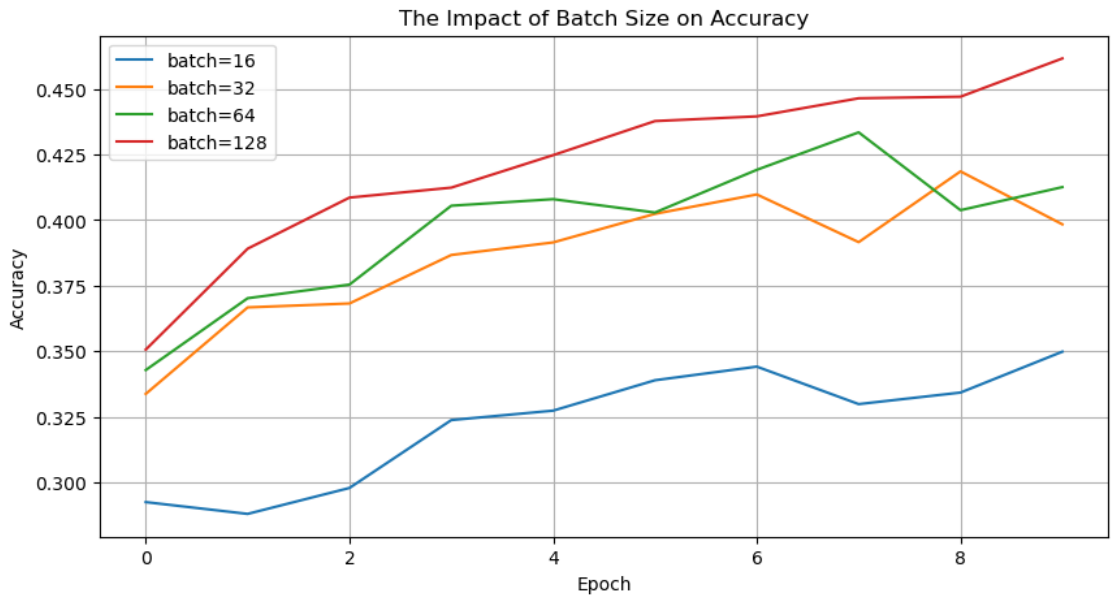
```
    train_loader = DataLoader(
        train_dataset,
        batch_size=batch_size,
        shuffle=True
    )

    test_loader = DataLoader(
        test_dataset,
        batch_size=batch_size,
        shuffle=False
    )

    model = MLP(
        hidden1=256,
        hidden2=128,
        dropout=0.3
    )

    losses, accuracies = train_model(
        model,
        train_loader,
        test_loader,
        epochs=10,
        lr=0.001
    )

    batch_results[batch_size] = {
        "losses": losses,
        "accuracies": accuracies,
        "final_accuracy": accuracies[-1]
    }
```

```
Training model with batch size = 16
Epoch 1: Loss=2.0339, Accuracy=0.2924
Epoch 2: Loss=1.9639, Accuracy=0.2879
Epoch 3: Loss=1.9523, Accuracy=0.2978
Epoch 4: Loss=1.9354, Accuracy=0.3237
Epoch 5: Loss=1.9178, Accuracy=0.3273
Epoch 6: Loss=1.9006, Accuracy=0.3389
Epoch 7: Loss=1.8949, Accuracy=0.3441
Epoch 8: Loss=1.8788, Accuracy=0.3298
Epoch 9: Loss=1.8797, Accuracy=0.3342
Epoch 10: Loss=1.8710, Accuracy=0.3498

Training model with batch size = 32
Epoch 1: Loss=1.9872, Accuracy=0.3337
```

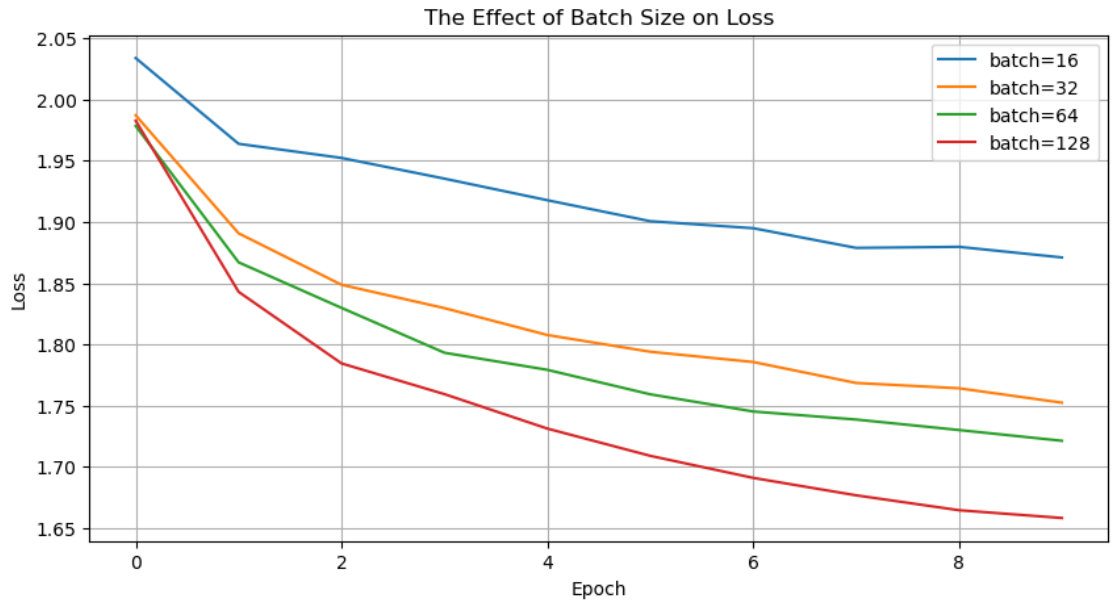
```
Epoch 2: Loss=1.8907, Accuracy=0.3667
Epoch 3: Loss=1.8487, Accuracy=0.3682
Epoch 4: Loss=1.8296, Accuracy=0.3867
Epoch 5: Loss=1.8077, Accuracy=0.3915
Epoch 6: Loss=1.7940, Accuracy=0.4024
Epoch 7: Loss=1.7856, Accuracy=0.4098
Epoch 8: Loss=1.7685, Accuracy=0.3916
Epoch 9: Loss=1.7641, Accuracy=0.4186
Epoch 10: Loss=1.7524, Accuracy=0.3984

Training model with batch size = 64
Epoch 1: Loss=1.9784, Accuracy=0.3428
Epoch 2: Loss=1.8670, Accuracy=0.3702
Epoch 3: Loss=1.8298, Accuracy=0.3754
Epoch 4: Loss=1.7932, Accuracy=0.4055
Epoch 5: Loss=1.7792, Accuracy=0.4080
Epoch 6: Loss=1.7592, Accuracy=0.4029
Epoch 7: Loss=1.7452, Accuracy=0.4192
Epoch 8: Loss=1.7386, Accuracy=0.4335
Epoch 9: Loss=1.7300, Accuracy=0.4038
Epoch 10: Loss=1.7213, Accuracy=0.4126

Training model with batch size = 128
Epoch 1: Loss=1.9827, Accuracy=0.3506
Epoch 2: Loss=1.8429, Accuracy=0.3891
Epoch 3: Loss=1.7845, Accuracy=0.4086
Epoch 4: Loss=1.7594, Accuracy=0.4124
Epoch 5: Loss=1.7312, Accuracy=0.4248
Epoch 6: Loss=1.7090, Accuracy=0.4378
Epoch 7: Loss=1.6909, Accuracy=0.4396
Epoch 8: Loss=1.6767, Accuracy=0.4465
Epoch 9: Loss=1.6645, Accuracy=0.4471
Epoch 10: Loss=1.6582, Accuracy=0.4617
```

[26]:
```python
plt.figure(figsize=(10, 5))

for batch_size in batch_sizes:

    plt.plot(
        batch_results[batch_size]["accuracies"],
        label=f"batch={batch_size}"
    )

plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.title("The Impact of Batch Size on Accuracy")
plt.legend()
```

```
plt.grid()
plt.show()
```



[27]:
```
plt.figure(figsize=(10, 5))

for batch_size in batch_sizes:

    plt.plot(
        batch_results[batch_size]["losses"],
        label=f"batch={batch_size}"
    )

plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("The Effect of Batch Size on Loss")
plt.legend()
plt.grid()
plt.show()
```



[28]:
```
batch_summary = []

for batch_size in batch_sizes:

    batch_summary.append({
        "Batch Size": batch_size,
        "Final Accuracy": batch_results[batch_size]["final_accuracy"]
    })

batch_df = pd.DataFrame(batch_summary)
batch_df
```

[28]:

	Batch Size	Final Accuracy
0	16	0.3498
1	32	0.3984
2	64	0.4126
3	128	0.4617

- Liczba neuronów: Zwiększenie liczby neuronów w warstwach ukrytych (do 512) pozytywnie wpłynęło na celność modelu.

[29]:
```
hidden_sizes = [64, 128, 256, 512]
hidden_results = {}
```

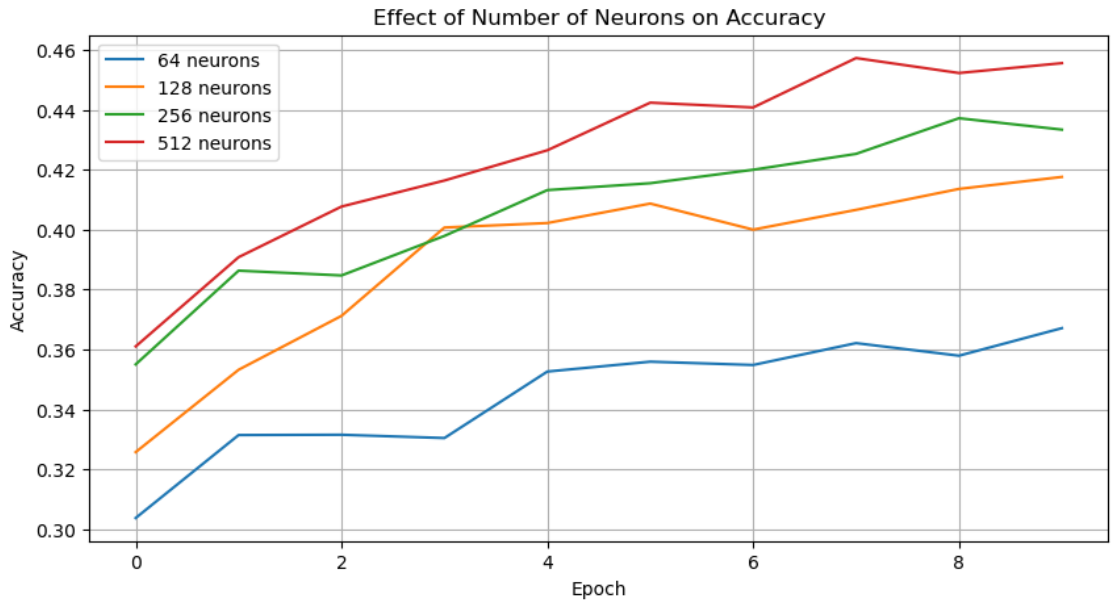
[30]:
```
for size in hidden_sizes:

    print(f"\nTraining model with hidden size = {size}")
```

```
    model = MLP(
        hidden1=size,
        hidden2=size // 2,
        dropout=0.3
    )

    losses, accuracies = train_model(
        model,
        train_loader,
        test_loader,
        epochs=10,
        lr=0.001
    )

    hidden_results[size] = {
        "losses": losses,
        "accuracies": accuracies,
        "final_accuracy": accuracies[-1]
    }
```

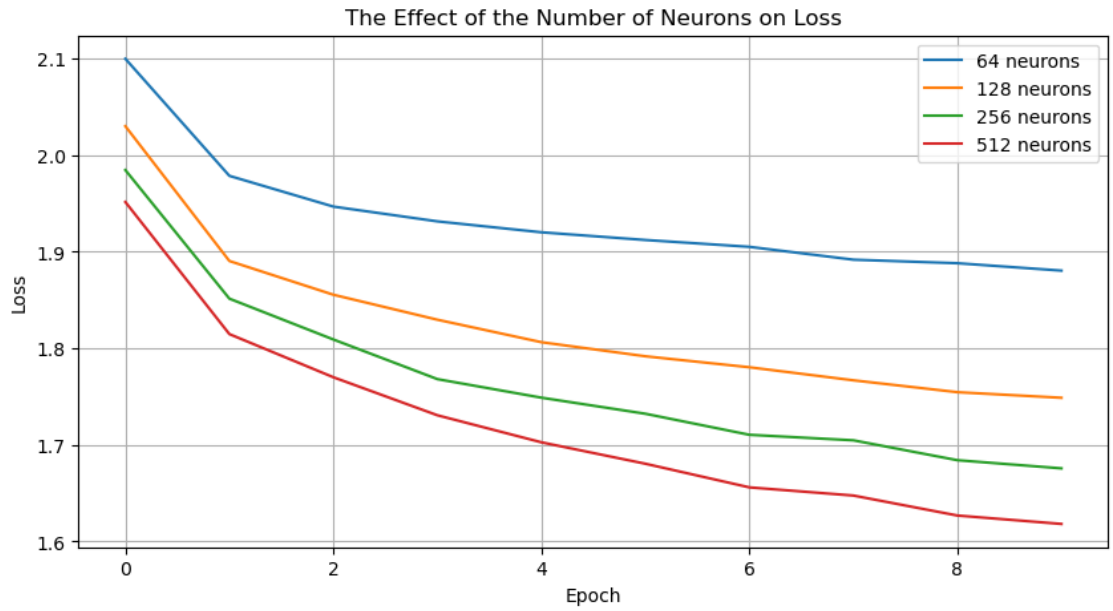
```
Training model with hidden size = 64
Epoch 1: Loss=2.0997, Accuracy=0.3037
Epoch 2: Loss=1.9785, Accuracy=0.3314
Epoch 3: Loss=1.9467, Accuracy=0.3315
Epoch 4: Loss=1.9314, Accuracy=0.3304
Epoch 5: Loss=1.9201, Accuracy=0.3526
Epoch 6: Loss=1.9121, Accuracy=0.3559
Epoch 7: Loss=1.9050, Accuracy=0.3548
Epoch 8: Loss=1.8917, Accuracy=0.3621
Epoch 9: Loss=1.8881, Accuracy=0.3579
Epoch 10: Loss=1.8804, Accuracy=0.3671

Training model with hidden size = 128
Epoch 1: Loss=2.0299, Accuracy=0.3257
Epoch 2: Loss=1.8904, Accuracy=0.3532
Epoch 3: Loss=1.8554, Accuracy=0.3712
Epoch 4: Loss=1.8296, Accuracy=0.4007
Epoch 5: Loss=1.8063, Accuracy=0.4022
Epoch 6: Loss=1.7917, Accuracy=0.4087
Epoch 7: Loss=1.7804, Accuracy=0.4000
Epoch 8: Loss=1.7668, Accuracy=0.4066
Epoch 9: Loss=1.7545, Accuracy=0.4136
Epoch 10: Loss=1.7488, Accuracy=0.4176

Training model with hidden size = 256
Epoch 1: Loss=1.9845, Accuracy=0.3550
```

```
Epoch 2: Loss=1.8514, Accuracy=0.3863
Epoch 3: Loss=1.8091, Accuracy=0.3847
Epoch 4: Loss=1.7681, Accuracy=0.3979
Epoch 5: Loss=1.7489, Accuracy=0.4132
Epoch 6: Loss=1.7322, Accuracy=0.4155
Epoch 7: Loss=1.7104, Accuracy=0.4200
Epoch 8: Loss=1.7047, Accuracy=0.4253
Epoch 9: Loss=1.6841, Accuracy=0.4372
Epoch 10: Loss=1.6757, Accuracy=0.4334

Training model with hidden size = 512
Epoch 1: Loss=1.9514, Accuracy=0.3610
Epoch 2: Loss=1.8147, Accuracy=0.3908
Epoch 3: Loss=1.7700, Accuracy=0.4077
Epoch 4: Loss=1.7307, Accuracy=0.4164
Epoch 5: Loss=1.7027, Accuracy=0.4265
Epoch 6: Loss=1.6805, Accuracy=0.4424
Epoch 7: Loss=1.6560, Accuracy=0.4408
Epoch 8: Loss=1.6476, Accuracy=0.4573
Epoch 9: Loss=1.6269, Accuracy=0.4523
Epoch 10: Loss=1.6182, Accuracy=0.4556
```

[31]:
```python
plt.figure(figsize=(10, 5))

for size in hidden_sizes:

    plt.plot(
        hidden_results[size]["accuracies"],
        label=f"{size} neurons"
    )

plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.title("Effect of Number of Neurons on Accuracy")
plt.legend()
plt.grid()
plt.show()
```



[32]:
```
plt.figure(figsize=(10, 5))

for size in hidden_sizes:

    plt.plot(
        hidden_results[size]["losses"],
        label=f"{size} neurons"
    )

plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("The Effect of the Number of Neurons on Loss")
plt.legend()
plt.grid()
plt.show()
```



[33]:
```
hidden_summary = []

for size in hidden_sizes:

    hidden_summary.append({
        "Hidden Size": size,
        "Final Accuracy": hidden_results[size]["final_accuracy"]
    })

hidden_df = pd.DataFrame(hidden_summary)
hidden_df
```

[33]:

	Hidden Size	Final Accuracy
0	64	0.3671
1	128	0.4176
2	256	0.4334
3	512	0.4556

- Dropout: Model bez dropoutu (0.0) osiągnął najwyższą celność na zbiorze testowym w tym konkretnym przypadku, co sugeruje, że przy tej architekturze problem przeuczenia nie był dominujący

[34]:
```
dropout_values = [0.0, 0.3, 0.5]
dropout_results = {}
```

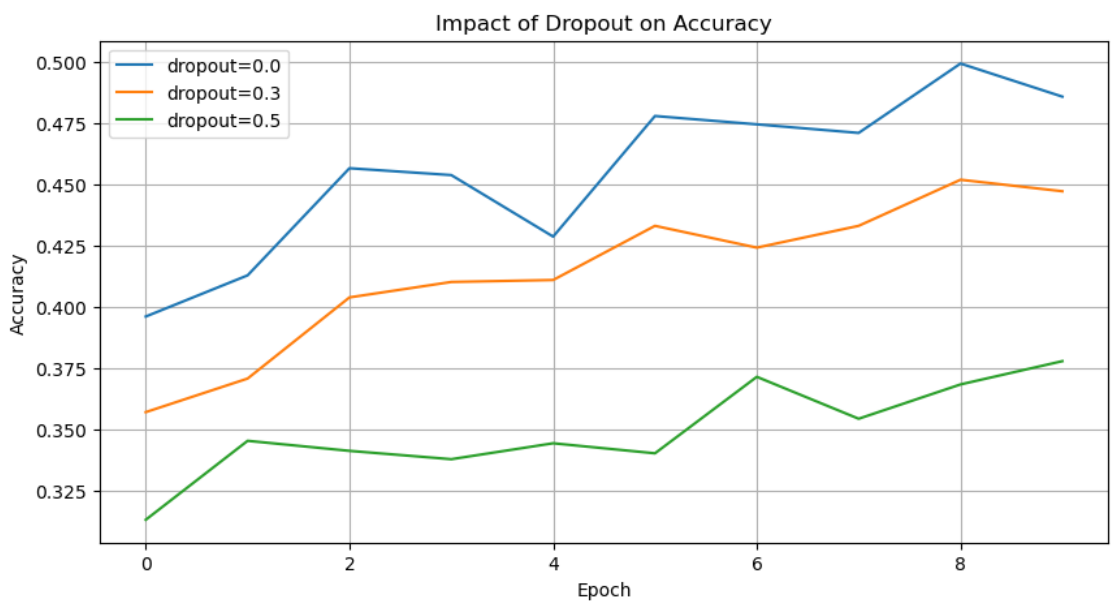
[39]:
```
for dropout in dropout_values:
```

```python
    print(f"\nTraining model with dropout={dropout}")
    model = MLP(
        hidden1=256,
        hidden2=128,
        dropout=dropout)

    losses, accuracies = train_model(
        model,
        train_loader,
        test_loader,
        epochs=10,
        lr=0.001
    )

    dropout_results[dropout] = {
        "losses": losses,
        "accuracies": accuracies,
        "final_accuracy": accuracies[-1]
    }
```

```
Training model with dropout=0.0
Epoch 1: Loss=1.8678, Accuracy=0.3940
Epoch 2: Loss=1.6681, Accuracy=0.4288
Epoch 3: Loss=1.5843, Accuracy=0.4356
Epoch 4: Loss=1.5335, Accuracy=0.4558
Epoch 5: Loss=1.4884, Accuracy=0.4713
Epoch 6: Loss=1.4553, Accuracy=0.4647
Epoch 7: Loss=1.4231, Accuracy=0.4859
Epoch 8: Loss=1.4014, Accuracy=0.4787
Epoch 9: Loss=1.3803, Accuracy=0.4913
Epoch 10: Loss=1.3567, Accuracy=0.4955

Training model with dropout=0.3
Epoch 1: Loss=1.9674, Accuracy=0.3688
Epoch 2: Loss=1.8398, Accuracy=0.3897
Epoch 3: Loss=1.7944, Accuracy=0.3824
Epoch 4: Loss=1.7581, Accuracy=0.3997
Epoch 5: Loss=1.7264, Accuracy=0.4183
Epoch 6: Loss=1.7190, Accuracy=0.4272
Epoch 7: Loss=1.6956, Accuracy=0.4285
Epoch 8: Loss=1.6871, Accuracy=0.4444
Epoch 9: Loss=1.6737, Accuracy=0.4408
Epoch 10: Loss=1.6658, Accuracy=0.4586

Training model with dropout=0.5
Epoch 1: Loss=2.0879, Accuracy=0.3243
Epoch 2: Loss=1.9744, Accuracy=0.3401
```

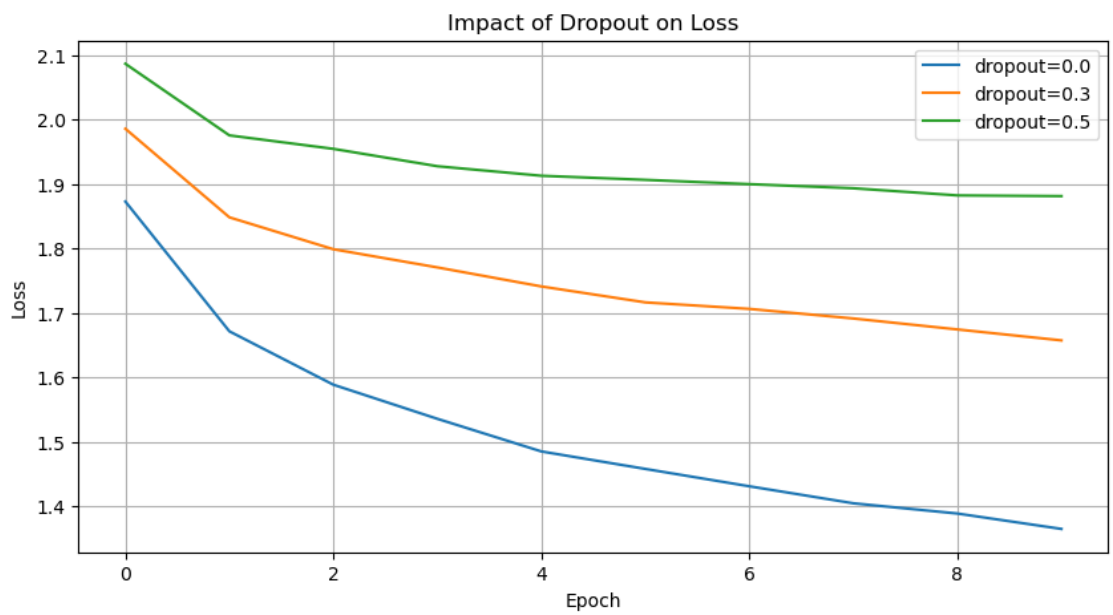
```
Epoch 3: Loss=1.9379, Accuracy=0.3512
Epoch 4: Loss=1.9158, Accuracy=0.3451
Epoch 5: Loss=1.9024, Accuracy=0.3688
Epoch 6: Loss=1.8901, Accuracy=0.3550
Epoch 7: Loss=1.8857, Accuracy=0.3649
Epoch 8: Loss=1.8795, Accuracy=0.3654
Epoch 9: Loss=1.8755, Accuracy=0.3673
Epoch 10: Loss=1.8610, Accuracy=0.3709
```

[36]:
```
plt.figure(figsize=(10, 5))

for dropout in dropout_values:

    plt.plot(
        dropout_results[dropout]["accuracies"],
        label=f"dropout={dropout}"
    )

plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.title("Impact of Dropout on Accuracy")
plt.legend()
plt.grid()
plt.show()
```



[37]:
```
plt.figure(figsize=(10, 5))
```

```
for dropout in dropout_values:

    plt.plot(
        dropout_results[dropout]["losses"],
        label=f"dropout={dropout}"
    )

plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Impact of Dropout on Loss")
plt.legend()
plt.grid()
plt.show()
```



[38]:
```
summary = []

for dropout in dropout_values:

    summary.append({
        "Dropout": dropout,
        "Final Accuracy": dropout_results[dropout]["final_accuracy"]
    })

dropout_df = pd.DataFrame(summary)
dropout_df
```

[38]:

	Dropout	Final Accuracy
0	0.0	0.4859
1	0.3	0.4473
2	0.5	0.3780

Wnioski

Prosta sieć MLP jest w stanie nauczyć się rozpoznawania obiektów na zdjęciach CIFAR-10 ze skutecznością znacznie wyższą niż losowa (która wynosiłaby 10%), jednak osiągnięty wynik ok. 45% pokazuje ograniczenia tej architektury w porównaniu do sieci konwolucyjnych (CNN). Kluczowym czynnikiem dla stabilności procesu uczenia okazał się odpowiedni dobór współczynnika uczenia oraz rozmiaru warstw ukrytych